

Network-Aware Multiway Join for MapReduce

Kenn Slagter¹, Ching-Hsien Hsu^{2,*}, Yeh-Ching Chung¹, and Jong Hyuk Park³

¹ Department of Computer Science, National Tsing Hua University
Hsinchu, Taiwan, R.O.C.

kennslagter@sslslab.cs.nthu.edu.tw, ychung@cs.nthu.edu.tw

² Department of Computer Science, Chung Hua University
Hsinchu, Taiwan, R.O.C.
chh@chu.edu.tw

³ Department of Computer Science and Engineering
Seoul National University of Science and Technology
Seoul, Korea
jhpark1@seoultech.ac.kr

Abstract. MapReduce is an effective tool for processing large amounts of data in parallel using a cluster of processors or computers. One common data processing task is the join operation, which combines two or more datasets based on values common to each. In this paper, we present a network aware multi-way join for MapReduce(NAMM) that improves performance by redistributing the workload amongst reducers. NAMM achieves this by redistributing tuples directly between reducers with an intelligent network aware algorithm. We show that our presented technique has significant potential to minimize the time required to join multiple datasets.

Keywords: MapReduce, Hadoop, Multiway Join, Workload Redistribution.

1 Introduction

MapReduce [1] is a flexible programming model proposed by Google for processing and creating data sets over a cluster of computers. The MapReduce model hides extraneous details inherent in distributed programming such as parallelization, fault tolerance, data distribution and load balancing within a library. This simplifies the process of writing distributed programs, which is an advantage MapReduce has over other distributed programming models such as MPI that requires the programmer to explicitly handle the data flow [2].

Programmers who use the MapReduce library need to write two functions a map function and a reduce function. The purpose of the map function is to take the input key/value pairs from an input source, process it and then outputs a set of intermediate key/value pairs. The intermediate key/value pairs it generates is then fed into a reduce function which processes the key/value pairs and then generates as output its own set of key/value pairs.

* Corresponding author.

Parallelism is achieved by running multiple map and reduce functions on multiple processors or machines. The intermediate key/value pairs produced by each of the map functions are partitioned so that intermediate key/value pairs that share the same key are all sent to the same reduce function to be processed.

Since its conception by Google the MapReduce model has inspired others to adopt its paradigm. One of the most well known adopters was Yahoo, who developed an open source implementation known as Hadoop [3] which operates under the Apache license. Hadoop is a Java-based implementation and by default runs on its own distributed file system (HDFS). Because Hadoop is open source, well documented and easy to use, the tool has gained prominence in the distributed programming community. For this reason, we use Hadoop as our reference platform for MapReduce in this paper.

The MapReduce model is effective at processing large amounts of data or datasets. A dataset is essentially a set of tuples stored in a file. In this paper, we look at one of the most common data processing operations called a join, which combines two or more datasets together based on some common value. There are many possible ways to implement a join. The efficiency of a join implementation depends on how many data sets there are and how large the data sets are. A MapReduce join can be implemented as a map-side join or a reduce-side join and multiple datasets may be handled either as successive two-way joins known as a cascade of joins or with a multiway join [4].

Multiway joins have certain advantages and disadvantages over cascade joins. First, it avoids considerable overhead since it does not to setup multiple jobs. Second, it can save space on the network since it does not need to store intermediate results. However, there are some drawbacks to multiway joins. When a multiway join is performed it needs to buffer tuples. This can lead to memory problems, especially if the data is skewed. Therefore, the number of datasets and size of datasets are limited by the memory resources available.

The main idea of NAMM is to improve processing time of a multiway join by redistributing the workload between reducers. The main contributions of our work are as follows. First, we present a model to redistribute tuples amongst reducers on the MapReduce framework for a multiway join. Second, we show how the NAMM redistribution algorithm can reduce job response times for a multiway join by considering network distance and reducer workload. Third, we compare our method to an alternative method, which does not take into account these factors.

The rest of this paper is organized as follows. Section 2 explains our research model and presents the proposed techniques on multiway joins and tuple redistribution. In Section 3, the simulation results and performance analysis are given to weigh the pros and cons of the proposed method. In Section 4, we discuss related work. Finally, the conclusion and future work are presented in Section 5.

2 Research Model

2.1 Network Model

The research model for this study is presented in Figure 1, which shows a network environment consisting of switches, racks and nodes. The two-level tree topology

shown in Fig 1(a) is a common network layout used by Hadoop. Each rack contains a set of servers (nodes) all interlinked by a switch. The racks themselves then uplink to a core switch or router. It is important to note that the total bandwidth between nodes on the same rack is much greater than that between nodes on different racks. The nodes are used to run map or reduce tasks as shown in Fig 1(b). In this paper, map tasks and reduce tasks are also referred to as mappers and reducers respectively.

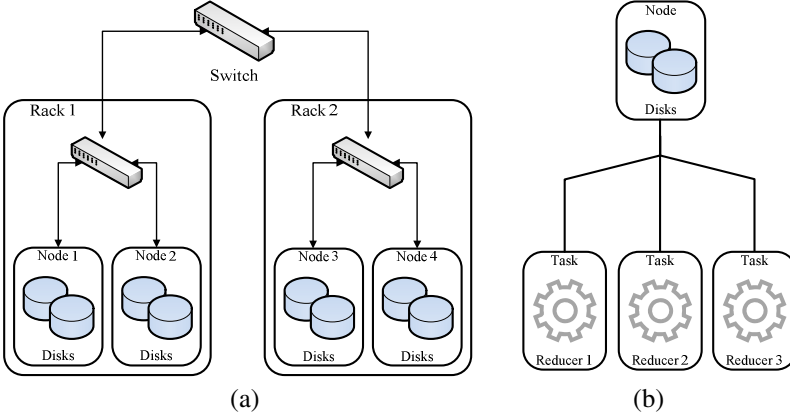


Fig. 1. Research model in this paper (a) a tree network consisting of racks and nodes (b) A node running a set of reduce tasks

2.2 Join Algorithms

Join algorithms have been studied extensively over the years, with many different variants existing for each type of algorithm. Many join algorithms in academia pre-date the invention of MapReduce, due to their ubiquitous use throughout the database community. The multiway join algorithm presented in this paper is a hybrid join based on pre-existing MapReduce join model for reduce-side joins and a hash-join which handles joins locally on the reducer.

2.2.1 Reduce-Side Join

Reduce-side joins are based on the MapReduce programming model which is composed of a map phase and a reduce phase. In the map phase, the datasets are read by a map function by each map task, one tuple at a time. The purpose of the map function is only to pre-process the tuples and sort them by the join key. Before tuples are partitioned based on their join key, they are tagged so that the reduce function can know which table the tuple originated from. The tuples are then sent to their respective reducer where they are to be joined. Each tuple is then joined by the reducer based on which table they came from.

2.2.2 Hash Join

A hash join is a traditional algorithm used by databases for joining two datasets together. A hash join consists of two distinct phases a ‘build’ phase and a ‘probe’ phase. In the build phase the smallest dataset is inserted into a in-memory hash table. In the probe phase the largest dataset is scanned and joined with the appropriate tuple(s) stored in the hash table.

2.2.3 NAMM Multiway Join

In this section we present our proposed multiway join. The purpose of a multiway join is to join multiple datasets together. Our proposed join improves performance of the multiway join by redistributing the workload amongst reducers. Unlike other schemes that redistribute the workload using a distributed queue [5], our methodology redistributes the workload directly between reducers with help of a mediator service, as shown in fig 2.

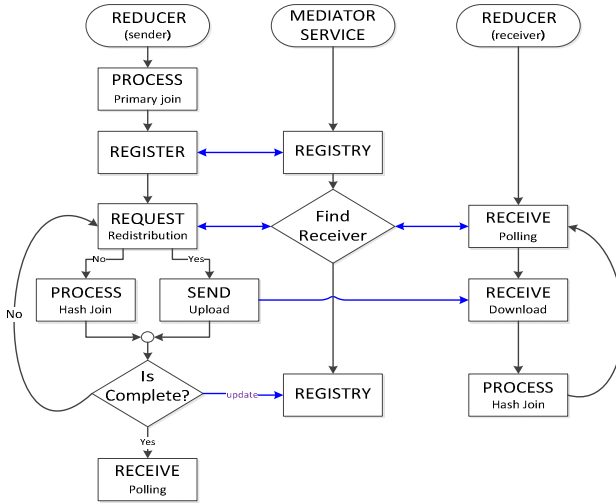


Fig. 2. Multiway Join Tuple Redistribution with NAMM

The reduce-side join and the hash join algorithms are both examples of a two-way join. Two-way joins are joins that involve only two tables. Multiway joins are joins involving more than two tables. The multiway join presented in this paper uses a reduce-side join to join the two largest datasets and a hash join on the reducer side to join that result with several smaller datasets. The two largest datasets are partitioned and sent to the various reducers using the typical reduce-side join mechanism.

Unlike the typical multiway join mechanism [6], the smaller datasets are sent to all the reducers. This can be done by duplicating tuples in the mapper phase so that a

copy is sent to each reducer or in Hadoop by using its distributed cache mechanism. Therefore, this method is appropriate in situations where the aggregate size of the smaller datasets can fit in the memory of each node used to execute a reduce task.

Once the mappers have sent the reducers all the tuples, the reducers are able to process the tuples. The two largest datasets are then joined with a traditional reduce-side join. After completing the initial primary join, the reducer registers with the mediator. The mediator is provided details from the reducer about the number of tuples it has, and details on which node it resides.

From the list of reducers already registered, the reducer will attempt to send a batch of tuples to another reducer. For the sake of clarity in this paper, we define two types of reducers, senders and receivers. Senders are those reducers that still have tuples to join and receivers are idle reducers that already processed their workload. The sender makes a request to the mediator to find a receiver on the network. It then prepares a batch of tuples to send from the initial primary join. The size of the batch depends on memory size of reducers and number of tuples being processed. The mediator then finds the reducer closest on the network in terms of network distance. If the mediator is unable to find a suitable receiver, the sender hash joins the batch of tuples it prepared instead. Network distance in this study is based on the same concept used in Hadoop [3] and is calculated based on the number of switches that exist between two nodes. There are three different scenarios that may occur in a data center. Given a node $n1$ on rack $r1$ in data center $d1$. This can be represented as $/d1/r1/n1$. Using this notation, here are the distances for the three scenarios:

- $\text{distance}(/d1/r1/n1, /d1/r1/n1) = 0$ (processes on the same node)
- $\text{distance}(/d1/r1/n1, /d1/r1/n2) = 2$ (different nodes on the same rack)
- $\text{distance}(/d1/r1/n1, /d1/r2/n3) = 4$ (nodes on different racks in the same data center)

Once a sender has sent all its tuples to a receiver it prepares another batch of tuples. If the sender has less tuples to process than other senders, it processes these tuples itself. Otherwise, the sender makes another redistribution request to the mediator.

3 Evaluation

3.1 Experiment Configuration

To evaluate the performance of the proposed technique, we implemented the NAMM multiway join method and tested its performance on a simulated MapReduce environment. We then evaluated its performance against a network unaware method. Overall, the network unaware method is the same as the NAMM method but sends its tuples to the first available receiver. We then tested both algorithms using various workloads using 200, 600 and 1000 million tuples. These workloads represent low loading (*LL*), medium loading (*ML*) and high loading (*HL*), respectively. For the sake of clarity, the experimental parameters used in this study are presented in Table 1.

Table 1. Experimental Parameters

Parameter	Definition	Description
LL	Low Loading	200M tuples
ML	Medium Loading	600M tuples
HL	High Loading	1000M tuples
n	n Loading	n M tuples

In order to test our proposed algorithm the MapReduce environment was setup to emulate a small cluster of computers. The cluster for our test environment consisted of two racks, with two nodes per rack and three reducers per node as shown in Fig 1. In total, 18 cases were used to test the performance of NAMM. These test cases are presented in Table 2.

Table 2. Test Cases

Rack		Rack 1						Rack 2					
Node		Node 1			Node 2			Node 3			Node 4		
Reducer		1	2	3	4	5	6	7	8	9	10	11	12
Test Case	1	ML	ML	ML	ML	ML	ML	ML	ML	ML	ML	ML	ML
	2	LL	HL	LL	LL	LL	LL	LL	LL	LL	LL	LL	LL
	3	ML	HL	ML	ML	ML	ML	ML	ML	ML	ML	ML	ML
	4	HL	LL	HL	LL	LL	LL	LL	LL	LL	LL	LL	LL
	5	LL	HL	LL	LL	HL	LL	LL	LL	LL	LL	LL	LL
	6	LL	HL	LL	LL	LL	LL	LL	LL	HL	LL	LL	LL
	7	HL	ML	HL	ML	ML	ML	ML	ML	ML	ML	ML	ML
	8	ML	HL	ML	ML	ML	ML	ML	ML	ML	ML	ML	ML
	9	ML	HL	ML	ML	ML	ML	ML	ML	HL	ML	ML	ML
	10	LL	LL	LL	HL	HL	HL	HL	HL	HL	HL	HL	HL
	11	LL	LL	LL	LL	LL	LL	HL	HL	HL	HL	HL	HL
	12	LL	LL	LL	LL	LL	LL	LL	LL	LL	HL	HL	HL
	13	ML	ML	ML	HL	HL	HL	HL	HL	HL	HL	HL	HL
	14	ML	ML	ML	ML	ML	ML	HL	HL	HL	HL	HL	HL
	15	ML	ML	ML	ML	ML	ML	ML	ML	HL	HL	HL	HL
	16	0	100	200	300	400	500	600	700	800	900	1000	1100
	17	1100	1000	900	800	700	600	500	400	300	200	100	0
	18	LL	HL	LL	HL	LL	HL	LL	HL	LL	HL	LL	HL

3.2 Experiment Results

In this paper, we compare five different redistribution methods using the test cases from Table 2. The redistribution methods considered in this study use a combination of the redistribution features shown in Table 3.

Table 3. Redistribution Features

Undistributed	no tuple redistribution
Network Unaware	network distance between reducers not considered
Network Aware	network distance between reducers considered
SingleSend	reducers redistribute tuples to only one receiver each hash join
Multisend	reducer with heaviest load attempts to redistribute tuples to multiple receivers before executing a hash join

The workload of a reducer is the total number of reduce-side joins and hash joins performed by that reducer. A MapReduce job does not complete until all mappers and reducers have completed their tasks. Therefore, the redistribution method that has the best performance is the redistribution method whose worst-case reducer has the least number of joins. We consider the worst-case reducer to be whichever reducer performed the most number of joins.

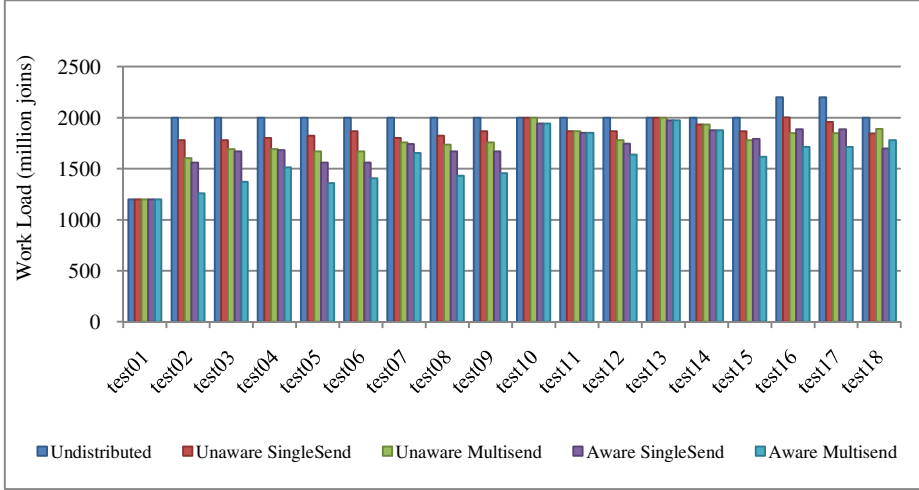


Fig. 3. The results for workload distribution on the worst-case reducer

In Fig 3, we compare the performance of the worst-case reducer for each redistribution method. In majority of test cases NAMM using network aware and multisend methods is able to significantly reduce the workload on each reducer, compared to other redistribution techniques covered in this study. The efficiency of NAMM is more apparent in test cases when the workload among reducers is less evenly distributed. In test case 1, all the reducers had exactly the same workload and consequently no redistribution takes place. In test case 13, and test case 14 the workload was distributed amongst reducer in such a fashion that there was no opportunity for multiple batch sends to occur. Test case 18 shows that in some instances multiple tuple redistributions from the same reducer can interfere with the overall time taken to complete a job. Overall, NAMM technique was able to improve workload redistribution by up to 12% when using the single send technique, and up to 21% improvement when compared to other network unaware methods assessed in this study.

4 Related Work

Joins have been studied in detail by many sources. Investigations and descriptions of the various joins have been collated by other works [6] [9]. One such work that discusses handling joins using a mediator over a network is presented by [7]. This system employs a balanced network utilization metric to optimize the use of all network

paths in a global-scale database federation. It uses a metric that allows algorithms to exploit excess capacity in the network, while avoiding narrow, long-haul paths. A work similar to our paper is presented by [5] uses a distributed queue [8] rather than using peerwise network connections to perform multiway joins and does not take into account network distance when redistributing tuples.

5 Conclusion and Future Work

In this paper, a network aware multiway MapReduce join (NAMM) technique is presented for redistributing workload for MapReduce. The simulation results show that NAMM can significantly improve tuple redistribution with Mutiway Joins for MapReduce applications. NAMM's has shown up to 21% improvement over other network unaware methods.

NAMM is designed for users who intend to use to perform a multiway join between two large datasets and several smaller datasets with MapReduce. In future work it would be desirable to explore how this system could be extended to handle more than two large datasets and how to improve its performance on different network topologies or hardware configurations. We leave these tasks for future work.

References

- [1] Dean, J., Ghemawat, S.: MapReduce: simplified data processing on large clusters. *Commun. ACM* 51, 107–113 (2008)
- [2] Hoefler, T., Lumsdaine, A., Dongarra, J.: Towards Efficient MapReduce Using MPI. In: Ropo, M., Westerholm, J., Dongarra, J. (eds.) *PVM/MPI. LNCS*, vol. 5759, pp. 240–249. Springer, Heidelberg (2009)
- [3] White, T.: *Hadoop the definitive guide*, 2nd edn. O'Reilly, Sebastopol (2010)
- [4] Afrati, F.N., Ullman, J.D.: Optimizing Multiway Joins in a Map-Reduce Environment. *IEEE Transactions on Knowledge and Data Engineering* 23, 1282–1298 (2011)
- [5] Lynden, S., Tanimura, Y., Kojima, I., Matono, A.: Dynamic Data Redistribution for MapReduce Joins. In: *2011 IEEE Third International Conference on Cloud Computing Technology and Science (CloudCom)*, pp. 717–723 (2011)
- [6] Chandar, J.: *Join Algorithms using Map/Reduce*, Master of Science, School of Informatics, University of Edinburgh (2010)
- [7] Wang, X., Burns, R., Terzis, A., Deshpande, A.: Network-aware join processing in global-scale database federations. In: *IEEE 24th International Conference on Data Engineering, ICDE 2008*, pp. 586–595 (2008)
- [8] Hunt, P., Konar, M., Junqueira, F.P., Reed, B.: *ZooKeeper: Wait-free coordination for Internet-scale systems*. In: *USENIX ATC* (2010)
- [9] Palla, K.: *A Comparative Analysis of Join Algorithms Using the Hadoop Map/Reduce Framework*, Master of Science, School of Informatics, University of Edinburgh (2009)